

ReflectionPad3d#

[https://pytorch.org/docs/stable/generated/torch.nn.modules.padding.Reflection](https://pytorch.org/docs/stable/generated/torch.nn.modules.padding.ReflectionPad3d.html)

Description:

Pads the input tensor using the reflection of the input boundary. For N-dimensional padding, use `torch.nn.functional.pad()`. padding (int, tuple) – the size of the padding. If is int, uses the same padding in all boundaries. If a 6-tuple, uses (padding_left\text{padding_left}padding_left, padding_right\text{padding_right}padding_right, padding_top\text{padding_top}padding_top, padding_bottom\text{padding_bottom}padding_bottom, padding_front\text{padding_front}padding_front, padding_back\text{padding_back}padding_back) Note that padding size should be less than the corresponding input dimension. Input: (N,C,D_{in},H_{in},W_{in})(N, C, D_{in}, H_{in}, W_{in})(N,C,D_{in},H_{in},W_{in}) or (C,D_{in},H_{in},W_{in})(C, D_{in}, H_{in}, W_{in})(C,D_{in},H_{in},W_{in}). Output: (N,C,D_{out},H_{out},W_{out})(N, C, D_{out}, H_{out}, W_{out})(N,C,D_{out},H_{out},W_{out}) or (C,D_{out},H_{out},W_{out})(C, D_{out}, H_{out}, W_{out})(C,D_{out},H_{out},W_{out}), where

Function Signature

```
class torch.nn.modules.padding.ReflectionPad3d(padding)  
[source]#
```

Parameters

Shape:

Code Examples

```
>>> m = nn.ReflectionPad3d(1)
>>> input = torch.arange(8, dtype=torch.float).reshape(1, 1, 2,
2, 2)
>>> m(input)
tensor([[[[ [7., 6., 7., 6.],
            [5., 4., 5., 4.],
            [7., 6., 7., 6.],
            [5., 4., 5., 4.]],
          [[3., 2., 3., 2.],
            [1., 0., 1., 0.],
            [3., 2., 3., 2.],
            [1., 0., 1., 0.]],
          [[7., 6., 7., 6.],
            [5., 4., 5., 4.],
            [7., 6., 7., 6.],
            [5., 4., 5., 4.]],
          [[3., 2., 3., 2.],
            [1., 0., 1., 0.],
            [3., 2., 3., 2.],
            [1., 0., 1., 0.]]]])])
```

Detailed Documentation

Documentation

Pads the input tensor using the reflection of the input boundary.

For N-dimensional padding, use `torch.nn.functional.pad()`.

padding (int, tuple) – the size of the padding. If is int, uses the same padding in all boundaries. If a 6-tuple, uses (padding_left\text{padding_left}padding_left, padding_right\text{padding_right}padding_right, padding_top\text{padding_top}padding_top, padding_bottom\text{padding_bottom}padding_bottom, padding_front\text{padding_front}padding_front, padding_back\text{padding_back}padding_back) Note that padding size should be less than the corresponding input dimension.

Input: (N,C,Din,Hin,Win)(N, C, D_{in}, H_{in}, W_{in})(N,C,Din,Hin,Win) or (C,Din,Hin,Win)(C, D_{in}, H_{in}, W_{in})(C,Din,Hin,Win).

Output: (N,C,Dout,Hout,Wout)(N, C, D_{out}, H_{out}, W_{out})(N,C,Dout,Hout,Wout) or (C,Dout,Hout,Wout)(C, D_{out}, H_{out}, W_{out})(C,Dout,Hout,Wout), where

$D_{out} = D_{in} + \text{padding_front} + \text{padding_back}$
 $D_{out} = D_{in} + \text{padding_front} + \text{padding_back}$

$H_{out} = H_{in} + \text{padding_top} + \text{padding_bottom}$
 $H_{out} = H_{in} + \text{padding_top} + \text{padding_bottom}$

$W_{out} = W_{in} + \text{padding_left} + \text{padding_right}$
 $W_{out} = W_{in} + \text{padding_left} + \text{padding_right}$